

گزارش کار پروژه

پروژه: پیاده‌سازی بازی دوز نیچران پیاده سازی با الگوریتم Minimax

درس: هوش مصنوعی

استاد: دکتر قوی‌دل

تهیه شده توسط علی فاضل نیا

دانشگاه شهید مدنی آذربایجان

دانشکده فناوری اطلاعات و مهندسی کامپیوتر

بهمن ماه ۱۴۰۳



باسمه تعالی

هدف

در این پروژه قصد داریم با استفاده از الگوریتم Minimax به پیاده‌سازی عاملی هوشمند که توانایی انجام بهینه بازی دوز نیچران را دارد بپردازیم. به این منظور از تعریف این الگوریتم و هرس آلفابتا استفاده شده.

شرح بازی و پیاده‌سازی آن

این بازی می‌تواند در ابعاد 3×3 تا 10×10 باشد. محاسبه امتیاز در پایان بازی زمانی که هیچ خانه خالی‌ای باقی نمانده هست انجام خواهد شد. اگرچه که در پیاده‌سازی الگوریتم Minimax نیاز به محاسبه امتیازها در هر مرحله و حالت داریم که آنها به طور جداگانه محاسبه خواهند شد. روش محاسبه امتیازها در این بازی به این صورت است که تمامی سطرها، ستون‌ها و قطرهای (اعم از تمامی قطرهای فرعی و اصلی) بررسی شده و در صورتی که در این خطها توالی‌های حرکات X (کاربر) و O (عامل) بیشتر از سه باشند حساب شده و در فرمول $2n-5$ جایگذاری می‌شوند که n در آن هر توالی محاسبه شده از حرکات است. این فرمول امتیازات این توالی‌ها رو محاسبه کرده و با جمع این امتیازها می‌توان به امتیاز نهایی برای مشخص کردن برنده و بازنده رسید. در ابتدای بازی کاربر باید حرکت اول رو انجام دهد و در ادامه به ترتیب عامل و کاربر به انجام حرکت خواهند پرداخت. همانطور که گفته شد برای هوشمند سازی عامل از الگوریتم Minimax استفاده شده است. ولی به این دلیل که این الگوریتم زمان زیادی برای محاسبه می‌گیرد از روش‌های زیر برای کمتر کردن زمان آن استفاده شده است:

(1) جستجوی محدود به عمق

(2) هرس آلفا بتا

(3) استفاده از متود Transposition Table

دلیل زمان زیاد این الگوریتم بررسی تمام حالات و خانه‌ها هست. به این منظور در مرحله اول از محدود کردن عمق آن استفاده شده تا این الگوریتم تا آخرین حالت ممکن را بررسی نکند و تا یک عمق مشخص که در همان ابتدا به آن داده شده به بررسی حالات بپردازد. نکته مهم این است که به قدری کم باید باشد که به دقت الگوریتم آسیبی نرساند لذا در این پیاده‌سازی عمق الگوریتم کمی زیاده در نظر گرفته شده است. در مرحله دوم با پیاده سازی هرس آلفا بتا به حذف شاخه‌هایی خواهیم پرداخت که محاسبه آنها کاملاً بی‌فایده است. این باعث کم شدن محاسبه و در نتیجه کمتر شده زمان اجرایی الگوریتم می‌شود. در مرحله سوم به ذخیره سازی حالت‌های از قبل محاسبه شده خواهیم پرداخت. به این صورت که در هربار محاسبه حالت‌ها امتیاز آن حالت و

حرکت بهینه در دیکشنری‌ای ذخیره شده و در بررسی حالت‌های بعدی اگرچه حالت مشابه دیده شد بدون پیشروی بیشتر، مقادیر ذخیره شده در دیکشنری برگردانده خواهد شد. برای بیشتر موثر بودن این روش از ذخیره نتایج حالت‌های هر بازی در یک فایل `txt` و استفاده از اطلاعات آن در بازی بعدی بهره برده‌ایم.

این فایل‌های `text` با نام `xtransposition_table{size}.text` برای عامل `min` و `otransposition_table{size}.text` برای عامل `max` که `size` در آن همان ابعاد برد بازی است، در محل ذخیره کد، ذخیره خواهند شد و هربار که برنامه اجرا می‌شود این فایل‌ها خوانده شده و به دیکشنری تبدیل شده و از آن در الگوریتم استفاده خواهد شد.

برای پیاده سازی برد در محیط برنامه نویسی از دیتافریم فریم‌ورک `Pandas` استفاده شده است. به این منظور که سطرها با لیبل حروف کوچک انگلیسی (`a-z`) و لیبل ستون‌ها با اعداد (`1-10`) مشخص گردند. ولی برای بررسی سطرها، ستون‌ها و قطرهای به منظور ساده‌تر شدن از آرایه‌های `Numpy` استفاده شده است.

به منظور ارتباط هرچه بهتر کاربر با محیط بازی و آسان‌تر شدن آن به پیاده‌سازی یک رابطه گرافیکی (`GUI`) با استفاده از فریم‌ورک `Pygame` پرداختیم. به این منظور وقتی که کد ران می‌شود در مرحله اول از کاربر ابعاد جدول بازی را خواسته شده. در مرحله بعدی که مرحله اصلی است پنجره بازی نمایش داده می‌شود که کاربر در آن به رقابت با عامل خواهد پرداخت. زمانی که بازی به اتمام رسد با یک ثانیه مکس، پنجره بازی بسته خواهد شد و پنجره‌ای برای نمایش نتیجه نمایش داده خواهد شد. در این پنجره امتیازهای نهایی کاربر، عامل، نتیجه بازی طبق امتیازهای نهایی و برد پایانی بازی نمایش داده خواهد شد.